

Panduan Pengembangan Aplikasi Android

Last Updated: June 3, 2026

1. Peran Aplikasi Android

Aplikasi Android bertindak sebagai antarmuka (dashboard) dan pengelola pengguna (user management) secara mandiri. **Master ESP32 TIDAK** lagi menangani autentikasi, login, atau manajemen user.

Tanggung jawab aplikasi Android:

1. **Autentikasi Lokal/Cloud:** Menyimpan data user (SuperAdmin, Admin, Staff) di database lokal Android (misal SQLite/Room) atau cloud (Firebase).
 2. **Dashboard Monitoring:** Memanggil `GET /api/slaves` ke Master ESP32 secara berkala (polling) untuk melihat status setiap excavator.
 3. **Pengiriman Perintah:** Mengirim `POST /api/command` untuk menambah waktu, pause, resume, dll.
 4. **Pengaturan Harga (Packages):** Melakukan update tarif timer (harga per paket durasi) langsung dari Dashboard Android via `POST /api/packages/update`.
 5. **Pembatasan Akses (Role Based):** Menampilkan/menyembunyikan tombol fitur (seperti Reset Pendapatan & Pengaturan Harga) berdasarkan Role user yang sedang login di Android.
-

2. Koneksi ke Master ESP32

- **Wi-Fi SSID:** `ExcavatorMaster`
- **Password:** `12345678`
- **Base URL Master API:** `http://192.168.4.1`

Catatan: Pastikan aplikasi Android mengizinkan *cleartext traffic* (HTTP biasa) di `AndroidManifest.xml` (`android:usesCleartextTraffic="true"`).

3. Integrasi Endpoint (Tanpa Token)

Semua endpoint di Master ESP32 bersifat **terbuka (Open Access)**. Tidak ada validasi bearer token atau password di sisi ESP32. Aplikasi Android bertanggung jawab penuh untuk mencegah user biasa (Staff) menekan tombol Admin.

Polling Status (Setiap 3 Detik)

Gunakan Retrofit/OkHttp untuk memanggil endpoint berikut:

- `GET /api/slaves` : Menampilkan daftar device, sisa waktu (`rem`), dan status (`RUNNING` , `LOCKED` , dll).
- `GET /api/history` : (Opsional) Mengambil data histori total bermain.
- `GET /api/revenue` : (Khusus Admin) Mengambil data pendapatan Rupiah.

Pengiriman Command

Ketika Staff menekan tombol Set 5 Menit di EXC-01:

1. Aplikasi mengirim JSON: `{"id": 1, "cmd": "ADD_TIME", "val": 5}` ke `POST /api/command`.
2. Master mem-forward ke Slave EXC-01.

3. Master merespon dengan status berhasil.

Reset Data (Khusus Admin)

Admin dapat menekan tombol Reset:

- `POST /api/history/reset` (Body kosong)
- `POST /api/revenue/reset` (Body kosong)
- `POST /api/reset-all` (Body kosong)

Pengaturan Harga Timer (Packages)

Karena harga timer diatur dari Dashboard Android, Admin memerlukan halaman **Settings / Pengaturan Harga**:

1. Saat halaman Settings dibuka, panggil `GET /api/packages` untuk menampilkan daftar harga saat ini.
2. Ketika Admin mengubah harga dan menekan "Simpan", kirim JSON: `{"id": 0, "priceIDR": 10000}` ke `POST /api/packages/update`.
3. Lakukan iterasi update untuk setiap paket yang diubah.

4. Aturan Bisnis & Logika Waktu (PENTING)

Untuk mencegah kebingungan Developer Android saat membangun UI dan mengintegrasikan API, perhatikan **aturan mutlak** berikut:

A. Sistem Paket Durasi (Bukan Tarif Per-Menit)

Sistem Master ESP32 sudah **dikunci secara sistem** menggunakan 7 slot "Paket Durasi", yaitu: `1 menit, 2 menit, 3 menit, 5 menit, 10 menit, 30 menit, dan 60 menit`.

- Admin **TIDAK** menetapkan 1 harga dasar untuk "tarif per menit" (misalnya Rp 1.000/menit).
- Admin menetapkan **HARGA TOTAL** secara spesifik/grosir untuk *masing-masing paket*. (Contoh: 5 menit = Rp 5.000, tapi 60 menit = Rp 45.000 agar lebih murah).
- Developer Android **WAJIB** membuat 7 tombol tambah waktu yang tetap (statis) di layar utama Staff. Tidak boleh ada *input textfield* bebas untuk memasukkan angka acak seperti 7 menit atau 11 menit.

B. Konsekuensi Mengirim Durasi Bebas

Saat menambah waktu, aplikasi Android akan memanggil `POST /api/command` dengan JSON: `{"id": 1, "cmd": "ADD_TIME", "val": 5}`

Jika `val` yang dikirim adalah 5 (angka yang cocok dengan slot paket), Master ESP32 akan mengenali paket tersebut dan mengkalkulasi total pendapatan (*revenue*) ke rekap penghasilan secara akurat.

[!WARNING] Jika UI Android mengizinkan Staff mengirim angka di luar pakem (misal `val = 7`), maka Master ESP32 tidak akan mengenali paket tersebut. Waktu alat di lapangan tetap akan bertambah, **TETAPI PENDAPATAN RUPIAH TIDAK AKAN TERCATAT (Dihitung Rp 0)** karena tidak ada "Paket 7 Menit" di database Master.

C. Alur UI Pengaturan Harga

Di halaman **Settings**:

1. Android me-request `GET /api/packages`.
2. Master membalas dengan *array* 7 buah paket durasi.

3. UI Android merender 7 baris kolom tersebut, masing-masing dengan label durasi dan *textfield* rupiah yang bisa diedit Admin.
4. Ketika Admin klik Simpan, Android harus mem- `POST` secara iteratif ke `/api/packages/update` untuk masing-masing `id` paket yang harganya diubah.

5. State Color Mapping

Desain UI di Android direkomendasikan menggunakan mapping warna berikut berdasarkan field `state` dari `/api/slaves` :

State	Rekomendasi Warna	Icon
<code>RUNNING</code>	Hijau	
<code>PAUSED</code>	Oranye	
<code>LOCKED</code>	Abu-abu	
<code>ENDED</code>	Merah	
<code>OFFLINE</code>	Abu-abu pudar	

6. Contoh Konfigurasi Retrofit (Java)

```
import retrofit2.Call;
import retrofit2.http.Body;
import retrofit2.http.GET;
import retrofit2.http.POST;
import java.util.List;

public interface ExcavatorApi {
    @GET("api/slaves")
    Call<List<SlaveDto>> getSlaves();

    @POST("api/command")
    Call<CommandResponse> sendCommand(@Body CommandDto cmd);

    @GET("api/history")
    Call<List<HistoryDto>> getHistory();

    @GET("api/revenue")
    Call<List<RevenueDto>> getRevenue();

    @GET("api/packages")
    Call<List<PackageDto>> getPackages();

    @POST("api/packages/update")
    Call<StatusDto> updatePackage(@Body PackageUpdatedDto pkg);

    // Manajemen Slave
    @POST("api/edit_slave")
    Call<StatusDto> editSlave(@Body EditSlaveDto edit);

    @POST("api/delete_slave")
    Call<StatusDto> deleteSlave(@Body DeleteSlaveDto delete);

    // Fitur Reset
    @POST("api/history/reset")
    Call<StatusDto> resetHistory();

    @POST("api/revenue/reset")
```

```

Call<StatusDto> resetRevenue();

@POST("api/reset-all")
Call<StatusDto> resetAll();
}

```

Tidak perlu lagi interceptor untuk Bearer Token ke ESP32, karena semua autentikasi ditangani dan divalidasi 100% di dalam internal aplikasi Android.

7. HTTP Status Codes & Error Handling

Untuk memastikan aplikasi Android tidak *crash* dan dapat memberikan *feedback* UI yang jelas (seperti *Toast* atau *Dialog*), Master ESP32 menerapkan standarisasi HTTP Status Code berikut:

HTTP Status	Arti	Penanganan di Android
200 OK	Sukses.	Lanjutkan proses parsing JSON.
400 Bad Request	Input tidak valid / JSON rusak / ID sudah dipakai.	Tampilkan <code>response.body().error</code> ke layar.
404 Not Found	Slave tidak ditemukan di <i>registry</i> .	Beritahu admin bahwa Slave dengan ID tersebut belum terdaftar.
502 Bad Gateway	Slave sedang <i>Offline</i> atau koneksi radio terputus.	Master ESP32 online, tetapi gagal mem- <i>forward</i> instruksi ke Slave. Tampilkan indikator "Slave Offline".
503 Service Unavailable	Master ESP32 sedang sibuk.	Terjadi <i>race condition</i> di sisi Master. Aplikasi Android dapat melakukan <i>Auto-Retry</i> 1-2 detik kemudian.

Bentuk baku untuk balasan error (Status 4xx dan 5xx) adalah:

```

{
  "ok": 0,
  "error": "Pesan error spesifik dari sistem"
}

```

8. API Reference & DTO Schemas

Berikut adalah detail struktur JSON (*Request* / *Response*) untuk setiap *endpoint* beserta rekomendasi kelas model DTO di Java/Kotlin. Sebagai tambahan, untuk memudahkan *testing*, kami telah menyediakan:

1. **OpenAPI/Swagger Spec:** Tersedia di file `docs/openapi.yaml`.
2. **Postman Collection:** Tersedia di file `docs/Excavator_API_Postman_Collection.json`. *Import* file ini ke Postman untuk langsung mencoba semua API tanpa harus *coding* terlebih dahulu.

1. Daftar Slaves (GET /api/slaves)

Response:

```
[
  {
    "id": 1,
    "ip": "192.168.4.2",
    "mac": "48:3F:DA:00:11:22",
    "name": "EXC-01",
    "state": "RUNNING",
    "rem": 3562,
    "last_seen": 2
  }
]
```

Java DTO:

```
public class SlaveDto {
    public int id;
    public String ip;
    public String mac;
    public String name;
    public String state; // "LOCKED", "RUNNING", "PAUSED", "ENDED", "OFFLINE"
    public int rem;      // Sisa waktu dalam detik
    public int last_seen; // Detik berlalu sejak slave terakhir merespon
}
```

2. Kirim Perintah (POST /api/command)

Request Body:

```
{
  "id": 1,
  "cmd": "ADD_TIME",
  "val": 60
}
```

Note: `val` untuk `ADD_TIME` adalah dalam format *menit*.

Response:

```
{
  "ok": 1,
  "code": "OK",
  "rem": 3600,
  "state": "RUNNING"
}
```

Java DTO:

```
public class CommandDto {
    public int id;
    public String cmd; // "ADD_TIME", "PAUSE", "RESUME", "STOP", "IDENTIFY", "REBOOT"
    public int val;    // Parameter nilai (contoh: menit untuk ADD_TIME, 0 untuk lainnya)
}

public class CommandResponse {
    public int ok;
    public String code;
    public int rem;
}
```

```
    public String state;
}
```

3. Pendapatan & Paket

GET /api/revenue Response:

```
[
  { "id": 1, "revenue": 150000 },
  { "id": 2, "revenue": 50000 }
]
```

GET /api/packages Response:

```
[
  { "id": 0, "durationMin": 5, "priceIDR": 5000 },
  { "id": 1, "durationMin": 15, "priceIDR": 10000 }
]
```

POST /api/packages/update Request Body:

```
{
  "id": 0,
  "priceIDR": 6000
}
```

Java DTOs:

```
public class RevenueDto {
    public int id;
    public long revenue; // Rupiah
}

// Untuk menampung response List packages
public class PackageDto {
    public int id;           // ID Paket (0 - 3)
    public int durationMin;  // Menit
    public long priceIDR;    // Rupiah
}

// Untuk body request update package
public class PackageUpdateDto {
    public int id;
    public long priceIDR;
}
```

5. Riwayat Bermain (GET /api/history)

Response:

```
[
  { "id": 1, "sessions": 5, "totalSec": 18000 }
]
```

Java DTO:

```
public class HistoryDto {
    public int id;
    public int sessions; // Jumlah disewa
    public long totalSec; // Total durasi jalan (detik)
}
```

6. Manajemen Slave (edit_slave & delete_slave)

Digunakan untuk mengubah pengaturan Unit/Slave (mac address) di Master.

POST /api/edit_slave Request Body:

```
{
  "mac": "24:6F:28:XX:XX:XX",
  "id": 2
}
```

(Response berupa { "ok": 1 })

POST /api/delete_slave Request Body:

```
{
  "mac": "24:6F:28:XX:XX:XX"
}
```

(Response berupa { "ok": 1 })

Java DTOs:

```
public class EditSlaveDto {
    public String mac;
    public int id; // ID / Nomor urut baru
}

public class DeleteSlaveDto {
    public String mac;
}
```

7. Standard Success Response

Untuk endpoint reset (/api/reset-all , dll) atau update paket (/api/packages/update).

Response:

```
{
  "ok": 1
}
```

Java DTO:

```
public class StatusDto {
    public int ok;
    public String message; // (Optional)
}
```

8. Contoh Implementasi Kode (Java/Android)

Bagian ini berisi contoh kode konkrit untuk memanggil API Master ESP32 dari Android.

A. Konfigurasi Retrofit Client

Gunakan *timeout* yang cukup (misalnya 10-15 detik) karena Master harus mem-*forward* instruksi ke Slave melalui radio WiFi (bisa sedikit tertunda).

```
import java.util.concurrent.TimeUnit;
import okhttp3.OkHttpClient;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class ApiClient {
    private static final String BASE_URL = "http://192.168.4.1/"; // Pastikan diakhiri slash (/)
    private static Retrofit retrofit = null;

    public static ExcavatorApi getApi() {
        if (retrofit == null) {
            OkHttpClient client = new OkHttpClient.Builder()
                .connectTimeout(15, TimeUnit.SECONDS)
                .readTimeout(15, TimeUnit.SECONDS)
                .build();

            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
        return retrofit.create(ExcavatorApi.class);
    }
}
```

B. Memanggil Data Slaves (Dashboard Polling)

Dashboard harus me-`refresh` data `getSlaves` setiap beberapa detik secara asinkron.

```
public void fetchDashboardData() {
    ApiClient.getApi().getSlaves().enqueue(new retrofit2.Callback<List<SlaveDto>>() {
        @Override
        public void onResponse(Call<List<SlaveDto>> call, retrofit2.Response<List<SlaveDto>> response) {
            if (response.isSuccessful() && response.body() != null) {
                List<SlaveDto> slaves = response.body();
                // Update UI RecyclerView/Adapter Anda di sini
                // slaves.get(0).state -> "RUNNING", dll
            }
        }

        @Override
        public void onFailure(Call<List<SlaveDto>> call, Throwable t) {
            // Tangani error, tampilkan indikator "Master Offline"
        }
    });
}
```



```

    }
  });
}

```

C. Menambah Waktu (Command ADD_TIME)

Ingat kembali [Aturan Bisnis](#), pastikan `val` hanya berisi nilai yang sesuai slot paket.

```

public void addTimeForExcavator(int excavatorId, int durationMinutes) {
    CommandDto cmd = new CommandDto();
    cmd.id = excavatorId;
    cmd.cmd = "ADD_TIME";
    cmd.val = durationMinutes; // HARUS 1, 2, 3, 5, 10, 30, atau 60

    ApiClient.getApi().sendCommand(cmd).enqueue(new retrofit2.Callback<CommandResponse>() {
        @Override
        public void onResponse(Call<CommandResponse> call, retrofit2.Response<CommandResponse> response) {
            if (response.isSuccessful() && response.body() != null) {
                if (response.body().ok == 1) {
                    // Sukses menambah waktu
                    // Response body.rem akan berisi sisa waktu terbaru dalam detik
                } else {
                    // Tampilkan pesan error dari response.body().code
                }
            } else {
                // Tampilkan pesan kegagalan (misal Slave Offline)
            }
        }

        @Override
        public void onFailure(Call<CommandResponse> call, Throwable t) {
            // Tampilkan error (koneksi terputus/timeout)
        }
    });
}

```